

PDR (PAISA DO RE)

AI-Powered Alternative Credit Scoring Pipeline

Complete Technical Architecture, Workflow, and File System Deep-Dive
Prepared for Barclays Hack-O-Hire 2026

TECHNICAL OVERVIEW SPECIFICATION

Backend: FastAPI Server, SQLite3 (WAL Mode Enabled)
Frontend: React.js (Vite, Framer Motion, Chart.js, Lucide Icons)
ML Pipeline: XGBoost Classifier (NTC and MSME), Platt Calibration
XAI Layer: SHAP TreeExplainer, Local Ollama Integration (Mistral)

1. Project Overview

PDR (Paisa Do Re) is a comprehensive B2B SaaS platform specifically designed to score "credit-invisible" borrowers in India, such as New-to-Credit (NTC) individuals and Micro, Small, and Medium Enterprises (MSMEs). A massive portion of the Indian population (over 400 million individuals) lacks formal bureau credit histories (like CIBIL or Experian). As a result, traditional lenders rely on bureau records and reject these profiles outright or charge extremely high risk premiums. PDR solves this problem by using alternative data sources, machine learning, and explainable AI (XAI) to deliver regulatory-compliant, transparent, and accurate credit scoring.

Major Features of the Platform:

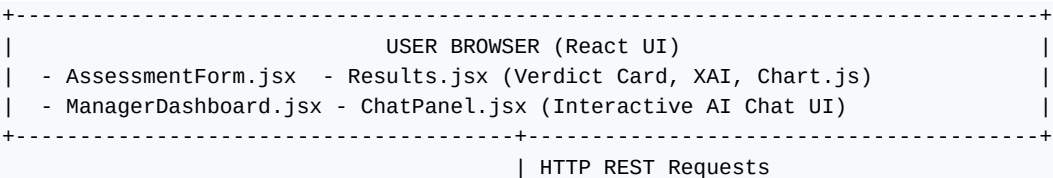
- Account Aggregator (AA) Integration: Consents, accesses, and ingests digitally signed, tamper-proof banking data directly via the Account Aggregator system.
- Dual Assessment Paths: Tailored evaluation engines and custom features for MSMEs and NTC individuals.
- Fraud & Forensic Audit Layer (Layer 2): Identifies circular loop trading (A->B->A cycles) using network graph algorithms (NetworkX) and flags manual ledger manipulation using Benford's Law anomaly analysis.
- Dual scoring approach: Combines a robust "pre-layer" rule engine (hard checks, demographic constraints) with a probabilistic XGBoost ML model to ensure high accuracy while catching fraudulent patterns instantly.
- SHAP-based Explainability (XAI): Extracts feature contribution values to provide exact regulatory "reason codes" determining why an application was approved or rejected.
- Interactive AI Credit Analyst Chatbot: Evaluates profiles, answers plain-English inquiries, generates formal decision letters, and runs what-if scenarios using local LLM models grounded in SQLite context.

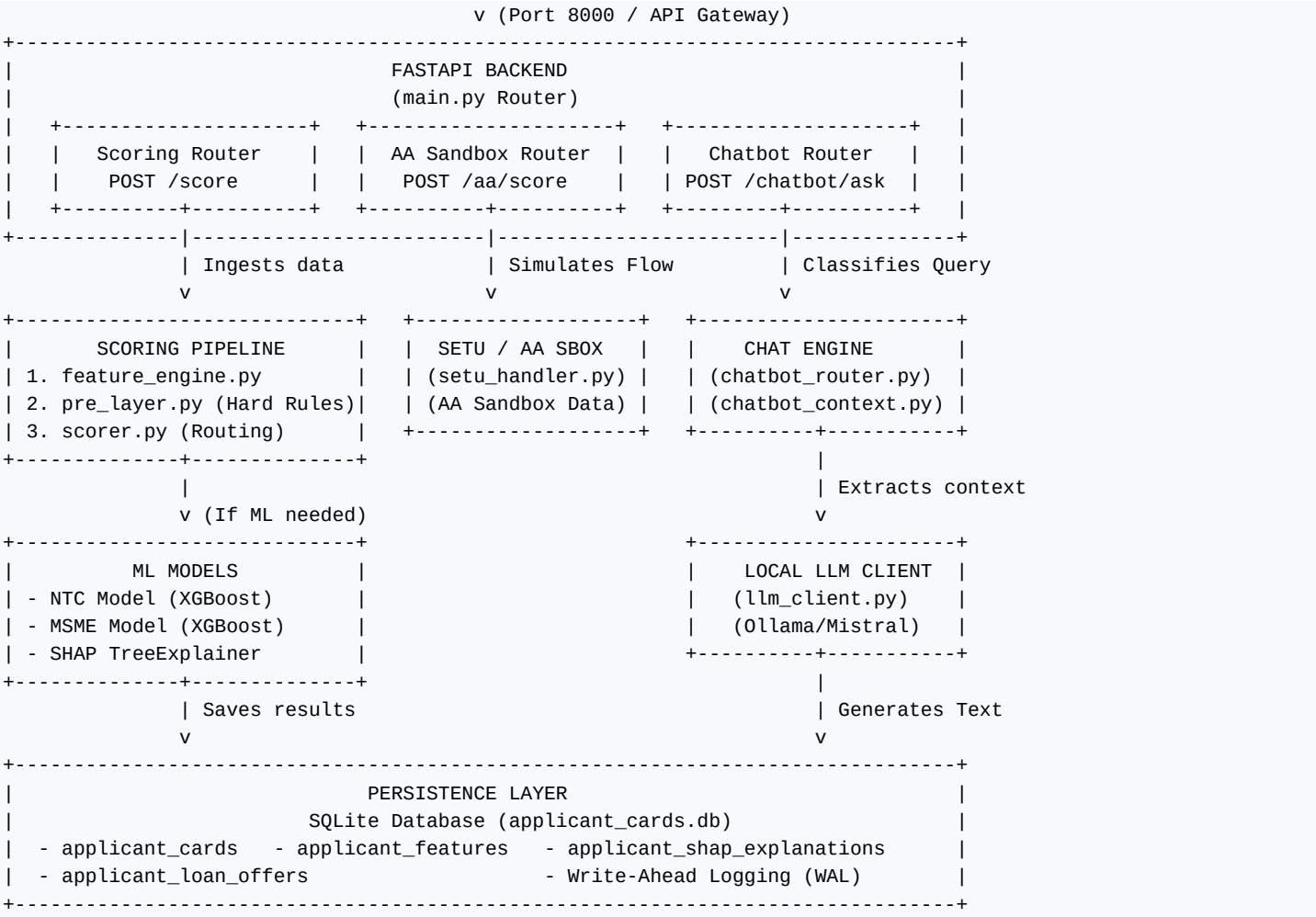
2. Project Architecture

The project follows a decoupling strategy separating frontend representation, backend orchestration, ML inference, and persistence. It is built as a three-tier web application:

1. Frontend (React + Vite): A high-performance, single-page application (SPA) styled using modular vanilla CSS. It features dynamic routing (React Router DOM v7), visual animations (Framer Motion), dashboard charts (Chart.js), and client-side PDF document compilation (jsPDF). Communication with the backend is done via standard HTTP/JSON.
2. Backend (FastAPI): High-throughput ASGI backend. It serves scoring pipelines, handles Account Aggregator simulation, seeds/manages the SQLite database, and manages chat requests to the local LLM. It includes a custom CORS middleware to allow local development connections.
3. Database Layer (SQLite3): Uses a local database (applicant_cards.db) with write-ahead-logging (WAL) to store scoring outputs, SHAP explanations, engineered features, and loan offers. WAL mode ensures multiple API worker threads can write concurrently without database locking.
4. ML & XAI Engine: Module-level loaded XGBoost models (saved as serialized joblib .pkl files) coupled with SHAP TreeExplainer objects. It executes real-time inference in less than 50ms.
5. LLM Client: Communicates with local Ollama service running a "mistral" model. Grounding prompts are constructed dynamically by combining router inputs with SQLite applicant context.

3. Architecture Diagram





4. Complete Working Flow

Here is the chronological operational sequence when the platform is loaded and run:

Step 1: System Boot & Initialization

- The FastAPI server is started (python main.py). Upon boot, the @app.on_event("startup") event listener triggers the database seeder (seed_chatbot_db). The function checks if applicant_cards.db exists, reads demo_users.json, and runs build_applicant_cards.py to insert fresh demo profiles, computed features, SHAP factors, and loan offers. This ensures the database is pre-populated.
- Concurrently, scorer.py loads the serialized XGBoost pickle models (ntc_credit_model.pkl and xgb_msme_raw.pkl) into memory and instantiates SHAP TreeExplainers to enable immediate, sub-50ms predictions.

Step 2: Profile Selection & Submission (Frontend)

- The loan officer opens http://localhost:5173/ and is presented with a landing page. They log in and select a pre-built demo profile (e.g., Ramesh Gowda, Priya Venkataraman, Deepak Malhotra) or upload a custom bank statement. Selecting a demo user auto-populates the React form fields in AssessmentForm.jsx. When the officer clicks "Submit for Assessment", a POST request is dispatched to http://localhost:8000/score containing the user_profile dictionary, transaction ledger list, and GST data.

Step 3: Feature Extraction

- The backend router receives the payload at POST /score and forwards it to compute_features() in feature_engine.py. This component processes transaction ledgers using Pandas, calculates cashflows, detects keywords like "ATM", "CASH", "RENT", "ELECTRICITY", "ECS BOUNCE", and parses telecom vintage and GST declarations to produce a standardized dictionary of

53 alternative credit features.

Step 4: Pre-Layer Gatekeeping

- The features dictionary is passed to `apply_pre_layer()` in `pre_layer.py`. It evaluates three priority levels of business rules. If a rule fires (e.g., circular loops with bounces, GST variance > 150%, or 5+ annual bounces), it bypasses the ML model entirely, assigning a hard grade (A, B, C, or E) and a primary decision reason.

Step 5: Machine Learning & SHAP (If pre-layer falls through)

- If no pre-layer rule fires, the application routes the applicant: NTC profiles go to `ntc_model`, and MSME profiles go to `msme_model`. The model returns a raw default probability. Then, the SHAP TreeExplainer computes Shapley values for the feature vector, identifying the top 5 risk or strength drivers. These features are mapped to human-readable text labels.

Step 6: Loan Policy and DB Logging

- Based on the default probability (or pre-layer decision), a letter grade (A to E) is assigned. The `_loan_offer()` helper calculates interest rate ranges, maximum loan amounts, and alternative products. Finally, `context_layer.py` executes insert queries to log everything in SQLite tables using WAL mode, and returns the response to the frontend.

Step 7: Render and Interactive Exploration

- The frontend receives the JSON response. `Results.jsx` mounts to show the decision, SHAP risk bars, and Chart.js cashflow charts. The loan officer can also open `ChatPanel.jsx` to ask natural language questions about the profile.

5. File-by-File Technical Deep-Dive

main.py (FastAPI Controller)

Purpose: The primary entry point and routing gateway for the backend API.

Contents: Defines the FastAPI instance, seeds the SQLite database on startup, registers CORS middleware, and defines 20+ REST endpoints (e.g., POST /score, POST /chatbot/ask, POST /setu/consent, GET /demo/{user_id}).

Downstream/Upstream: Called directly by the React frontend via HTTP requests. Imports scorer.py to score applicants, context_layer.py to fetch and update application status, chatbot_router.py to route chatbot queries, and setu_handler.py for Account Aggregator calls.

Contribution: Serves as the traffic controller, translating HTTP request bodies into structured inputs for the scoring and chat engines.

pre_layer.py (Rule Engine)

Purpose: Evaluates regulatory business rules and catches fraud/severe stress patterns before model scoring.

Contents: Hardcoded RBI and CIBIL-aligned threshold constants (e.g., T_BOUNCE_HARD_REJECT = 5, T_GST_VARIANCE_REJECT = 1.5) and the apply_pre_layer(features) function. It divides checks into Type 1 (Hard Rejections), Type 2 (Edge Case Protection/Auto-Approve), and Type 3 (Manual Review referrals).

Downstream/Upstream: Imported by scorer.py. Returns a decision tuple (grade, outcome, reason) or None if no rules fire.

Contribution: Protects the system against structural outliers and wash trading fraud patterns, satisfying regulatory audit standards.

scorer.py (Inference Pipeline)

Purpose: Orchestrates the scoring pipeline, model inference, and explainability calculations.

Contents: Loads NTC and MSME XGBoost model binaries, initializes SHAP TreeExplainers, maps raw SHAP outputs to English text, determines loan policy parameters, and contains the core score_user() function.

Downstream/Upstream: Imported by main.py. Imports feature_engine.py and pre_layer.py.

Contribution: Integrates feature engineering, hard rules, ML modeling, SHAP explanations, and loan policy calculations into a single API response.

feature_engine.py (Feature Engineering)

Purpose: Processes raw banking and profile data to construct the 53 feature vector.

Contents: compute_features(transactions, profile, gst_data) which implements keyword parsing, aggregations, and statistical calculations (e.g., coefficient of variation for EOD balance volatility, longest streaks of on-time utility bills, GSTR filing consistency counts, and cash withdrawal ratios).

Downstream/Upstream: Imported by scorer.py. Takes raw JSON arrays and returns structured key-value features.

Contribution: Translates noisy transactional datasets into clean, tabular signals ready for XGBoost models.

context_layer.py (Database access layer)

Purpose: Handles all SQLite database operations, including schema definition and CRUD functions.

Contents: SQLite DDL queries to create tables, index definitions, and database helper functions (init_database, save_applicant_card, fetch_applicant_card, search_applicants, update_applicant_status, delete_applicant_card).

Downstream/Upstream: Imported by main.py, chatbot_context.py, and build_applicant_cards.py.

Contribution: Isolates raw SQL commands, controls connection pooling, enforces foreign key checks, and logs scoring outcomes.

chatbot_router.py (NLP Classification)

Purpose: Classifies natural language questions from loan officers into structured queries.

Contents: Regex-based extraction patterns for applicant IDs (e.g., ntc_001, msme_003) and query classification rules. Routes messages into 7 query types: LOOKUP, EXPLANATION, COMPARISON, SCENARIO, DECISION_LETTER, RISK_ASSESSMENT, and AGGREGATE.

Downstream/Upstream: Imported by main.py. Fed by POST /chatbot/ask requests, returning a RoutedQuery dataclass.

Contribution: Pre-filters user questions to fetch only relevant data, preventing the LLM from hallucinating.

chatbot_context.py (Context Aggregation)

Purpose: Aggregates sqlite context records and builds system/user prompts for the LLM.

Contents: Prompt templates for the 7 query types. Implements build_prompt(routed_query, db_path) which fetches applicant cards, extracts features, and builds markdown text summaries of default risks, SHAP factors, and flags.

Downstream/Upstream: Imported by main.py. Imports context_layer.py functions to query database records.

Contribution: Injects grounding data into the LLM system prompt, forcing it to analyze the exact SQLite records.

llm_client.py (Ollama Client)

Purpose: Manages connections and generation requests to the local LLM.

Contents: call_ollama(system_prompt, user_prompt, query_type) which executes HTTP requests to Ollama API. Includes token limits by query type (e.g. 500 for decision letters, 80 for lookup) and retries on database grounding failures.

Downstream/Upstream: Imported by main.py. Calls the local Ollama HTTP endpoint.

Contribution: Integrates LLM text generation into the system, wrapping raw model outputs in clean layout formats.

response_formatter.py (UI String Wrapper)

Purpose: Wraps raw LLM text into formatted text boxes for the chatbot interface.

Contents: Formatter functions (e.g., _fmt_explanation, _fmt_comparison, _fmt_letter) that wrap text, add visual borders (= and - separators), and prepend status boxes like "[RISK]" or "[OK]".

Downstream/Upstream: Imported by main.py. Takes LLM outputs and writes them as CLI-style formatted text blocks.

Contribution: Formats chatbot responses, allowing loan officers to read structured comparisons and risk assessments.

AssessmentForm.jsx (React Form Controller)

Purpose: Renders forms for MSME and NTC profile creation, editing, and submission.

Contents: Forms for demographic data, business details, industry parameters, and synthetic ledger generators. Includes handlers to save current selections, parse CSV statement files, and post to /score backend routes.

Downstream/Upstream: Imports BankStatementUpload.jsx and Results.jsx. Routes submissions to the FastAPI backend.

Contribution: Manages borrower profile input, handles demo user profile switches, and updates dashboard states.

Results.jsx (React Assessment Render)

Purpose: Renders details, credit metrics, and alternative product offerings after scoring.

Contents: Verdict cards, interactive SHAP contribution bars, Chart.js cashflow charts, and action panels for the AI chatbot, manual overrides, and PDF report downloads.

Downstream/Upstream: Rendered dynamically inside AssessmentForm.jsx. Imports XaiTransparencySection.jsx, TransactionForensics.jsx, ChatPanel.jsx, and PdfReportGenerator.js.

Contribution: Translates backend JSON outputs into interactive risk charts, giving officers visual SHAP breakdowns.

PdfReportGenerator.js (Client PDF Compiler)

Purpose: Compiles a credit assessment report as a downloadable PDF file directly in the browser.

Contents: generateCreditDecisionPDF(scoringResult, applicantName, modelType) which calls jsPDF. Constructs page borders, tables, metric highlights, SHAP bars, and risk disclosures.

Downstream/Upstream: Imported and triggered by a button click in Results.jsx.

Contribution: Enables loan officers to export, print, and archive complete audit records for compliance files.

6. Feature-Level Workflows

Flow A: MSME Credit Assessment

1. Input: Loan officer selects Ramesh Gowda (Informal MSME) in AssessmentForm.jsx. The form fields populate with his data (Business Vintage: 36 months, GST filers consistency: 11/12, etc.).
2. Submit: Clicking "Submit" posts to `http://localhost:8000/score`.
3. Features: `feature_engine.py` extracts 53 features. It calculates revenue growth trend, revenue seasonality, operating cashflow ratios, client concentration ratios, and invoice payment delays from transactions.
4. Pre-layer: `pre_layer.py` checks rules. Since Ramesh has 0 bounces and 0 circular loops, he bypasses the pre-layer.
5. ML Model: `scorer.py` routes Ramesh to the MSME model (`xgb_msme_raw.pkl`). The model evaluates the 53 features and predicts a probability of default (PD) of 13.5%.
6. XAI: SHAP TreeExplainer calculates feature contributions. It finds that Ramesh's long business vintage and low cash withdrawal dependency are positive factors, while client concentration is a risk.
7. Loan Policy: The PD of 13.5% falls under Grade B (Approved with Conditions, interest rate: 14-20% p.a.).
8. Storage: `context_layer.py` saves these values to the SQLite tables.
9. Output: The backend returns the results. `Results.jsx` renders the details, `Chart.js` displays a cashflow trend, and the loan officer can download the report PDF.

Flow B: NTC Credit Assessment

1. Input: Officer selects Priya Venkataraman (Clean Salaried). Forms fill with phone vintage (2800 days), education tier, monthly annuity, assets, and dependents.
2. Submit: Clicking "Submit" posts to `http://localhost:8000/score`.
3. Features: `feature_engine.py` calculates rent wallet shares, utility payments consistency (11 months on-time), and EOD balance volatility (18%).
4. Pre-layer: Bypassed due to clean profile.
5. ML Model: Routed to the NTC model (`ntc_credit_model.pkl`), which yields a PD of 3.2%.
6. XAI: SHAP explainer identifies utility payment consistency and SIM vintage as key strengths.
7. Policy: Evaluates to Grade A (Approved, interest rate: 11-14% p.a., income multiplier: 20x).
8. Save & Render: Database tables are updated, and `Results.jsx` renders the screen with a green approval card.

Flow C: AI Credit Analyst Chatbot

1. Query: Loan officer types "Why was msme_002 rejected?" in ChatPanel.jsx and hits enter.
2. Route: `main.py` receives the question at `POST /chatbot/ask` and calls `route_query()`. The router extracts applicant ID "msme_002" and query type "EXPLANATION" using regex.
3. Context: `chatbot_context.py` calls `fetch_applicant_context()` which queries the database for msme_002. It fetches the SQLite records, extracts features (e.g. `p2p_circular_loop_flag = 1`, `bounces = 4`), and formatted SHAP factors.
4. Prompt: The builder constructs a system prompt (instructions to act as a strict credit analyst) and a user prompt containing the applicant profile, SHAP values, and red flags.
5. LLM Call: `call_ollama()` forwards prompts to Ollama (Port 11434, Mistral model). The LLM processes the context and writes a response: "Applicant was rejected due to circular trading and repeated payment bounces."
6. Formatter: `response_formatter.py` wraps the response in ASCII borders, adds applicant info, and lists SHAP factors.
7. Render: The frontend receives the formatted text and displays it in the chat console.

7. Database Schema & Tables

The project uses SQLite3 with Write-Ahead Logging (WAL) enabled (`PRAGMA journal_mode = WAL`). Foreign key constraints are enforced on connection (`PRAGMA foreign_keys = ON`). The database is structured into four main tables:

1. Table: applicant_cards

- Columns: id (INTEGER PK), applicant_id (TEXT UNIQUE), name (TEXT), city (TEXT), business_type (TEXT), grade (TEXT), outcome (TEXT), default_probability (REAL), decision_source (TEXT), primary_reason (TEXT), pre_layer_rule (TEXT), manager_remarks (TEXT), is_deleted (INTEGER), score_date (TEXT), created_at (TEXT), updated_at (TEXT).
- Purpose: Stores core application records, verdicts, and audit dates.

2. Table: applicant_features

- Columns: id (INTEGER PK), applicant_id (TEXT FK), feature_name (TEXT), feature_value (REAL), updated_at (TEXT).
- Indexes: Unique constraint on (applicant_id, feature_name). Index on applicant_id.
- Purpose: Stores the 53 feature values for each applicant for auditing.

3. Table: applicant_shap_explanations

- Columns: id (INTEGER PK), applicant_id (TEXT FK), rank (INTEGER), feature (TEXT), reason (TEXT), shap_value (REAL), direction (TEXT), impact (TEXT), updated_at (TEXT).
- Indexes: Unique constraint on (applicant_id, rank). Index on applicant_id.
- Purpose: Stores top 5 SHAP feature contributions for explainability.

4. Table: applicant_loan_offers

- Columns: id (INTEGER PK), applicant_id (TEXT FK UNIQUE), eligible (INTEGER), interest_rate_min (REAL), interest_rate_max (REAL), max_loan_amount (REAL), tenure_options_json (TEXT), recommended_product (TEXT), alternative_products_json (TEXT), updated_at (TEXT).
- Purpose: Stores calculated loan offers, tenures, and alternative options.

8. AI/ML Engine & SHAP Explainability

Models Architecture:

- PDR uses two XGBoost (eXtreme Gradient Boosting) tree classifier models trained on synthetic datasets simulating Indian borrower archetypes. Standard XGBoost output represents raw probability of default. The NTC model (ntc_credit_model.pkl) evaluates 12 behavioral features, and the MSME model (xgb_msme_raw.pkl) evaluates 15 operational features.

SHAP (Shapley Additive exPlanations) Integration:

- SHAP is based on cooperative game theory. It treats features as "players" in a game where the "payout" is the difference between the model's prediction and the average base prediction. SHAP calculates the marginal contribution of each feature to assign a Shapley value.

- Positive SHAP values (+0.18) increase the probability of default (representing a risk factor).

- Negative SHAP values (-0.22) decrease the probability of default (representing a strength factor).

- Explainers are instantiated at startup: ntc_explainer and msme_explainer are built using shap.TreeExplainer(model). During scoring, scorer.py runs explainer.shap_values(features_df) to get raw values. The values are sorted by absolute magnitude, and the top 5 features are saved to the database. Values are normalized relative to the maximum absolute feature impact to display visual percentage strength/risk bars on the UI.

9. Technologies & Frameworks Rationale

Below are the primary tools used in PDR and why they were chosen:

1. FastAPI (Backend): High-performance ASGI framework. Chosen for its automatic Swagger documentation, Pydantic data validation, and async support, allowing it to handle concurrent API requests easily.
2. React.js + Vite (Frontend SPA): React provides a component-based UI, while Vite delivers fast development builds. Vanilla CSS is used for custom styling without external library dependencies.
3. SQLite3 (Database): Lightweight, serverless relational database. Enabled in WAL (Write-Ahead Logging) mode, making it fast and easy to bundle directly inside the workspace without database configuration.
4. XGBoost (Machine Learning): Standard algorithm for tabular datasets, providing high training speeds and accuracy compared to traditional logistic regression models.
5. SHAP (Model Explainability): Industry-standard explainability framework, satisfying RBI audit requirements by delivering clear Shapley feature attribution values for credit decisions.
6. Ollama + Mistral (NLP Chatbot): Local LLM framework. Keeps sensitive financial data on local servers, ensuring data privacy while enabling conversational credit analysis.

10. Important Concepts Guide

- Account Aggregator (AA): An RBI-regulated financial data sharing framework. It allows borrowers to share digitally signed bank statements from their financial institutions with lenders in real-time, removing manual statement uploads.
- Benford's Law: An empirical law stating that the first digit of naturally occurring numerical datasets follows a logarithmic distribution (digit 1 appears ~30% of the time, digit 9 only ~4.6% of the time). PDR calculates statistical deviations from this curve to detect manual billing fraud.
- Circular Trading / P2P Loops: Fraud where entities cycle funds between related bank accounts (A->B->C->A) to inflate transaction volumes and look creditworthy. PDR builds a transaction graph using NetworkX to flag loop cycles.
- Platt Calibration / Sigmoid Scaling: Adjusts model predictions to match historical default rates, transforming XGBoost outputs into calibrated default probabilities.
- Days Past Due (DPD): A metric representing how many days a payment is late. PDR tracks utility bill and supplier invoice payment delays to assess borrower discipline.

11. Execution & Deployment Guide

How to start and run the project locally:

Prerequisites:

- Python 3.9+ and Node.js 18+ installed on your system.

Running the Backend API:

1. Open a terminal and navigate to the project root directory.
 2. Install required packages: `pip install fastapi uvicorn pydantic pandas numpy scikit-learn shap joblib requests`
 3. Start the FastAPI server: `python main.py`
- The database seeder will automatically initialize and populate `applicant_cards.db` on startup.
 - The API will run on `http://localhost:8000`.

Running the Frontend UI:

1. Open a second terminal and navigate to the `pdr-frontend` directory.
 2. Start the Vite development server: `npm run dev`
- The frontend will launch at `http://localhost:5173`.

Connecting the Chatbot to local LLM:

- Make sure Ollama is installed and running locally: `ollama run mistral`
- The backend calls Ollama at `http://localhost:11434` to answer conversational queries.

12. 3-Minute Viva / Interview Pitch

"Good morning. I am presenting PDR (Paisha Do Re), an AI-powered alternative credit scoring platform designed to evaluate credit-invisible borrowers in India, specifically NTC individuals and informal MSMEs who lack CIBIL or other formal bureau credit histories.

PDR uses a four-layer trust-gated pipeline to ingest alternative data (such as Account Aggregator bank statements, GST filing consistency, utility bill payment histories, and mobile number stability) and generates a transparent credit score.

The architecture is built with a React frontend and a FastAPI backend, using a local SQLite database in WAL mode. The scoring pipeline first applies a pre-layer rule engine to check for fraud or severe stress. For example, it builds network graphs using NetworkX to flag circular trading loops, and applies Benford's Law to check for ledger manipulation. If the applicant passes these checks, they are routed to NTC or MSME XGBoost classifiers, which output default probabilities.

To meet RBI requirements for explainability, we integrate SHAP to calculate Shapley values, providing exact reasons for every decision on the UI. Additionally, we built an AI Credit Analyst Chatbot. Using a local LLM grounded in the SQLite database, it allows loan officers to ask plain-English questions, generate decision letters, and run what-if scenarios without sharing sensitive borrower data with third-party models.

In short, PDR bridges the credit gap for underserved borrowers using robust machine learning, automated fraud detection, and explainable AI."